Assignment of Java

Important Instructions:

Objective:

The objective of this assignment is to test your understanding of core Java concepts related to classes, objects, and inheritance. You will answer theoretical questions and write Java code snippets to demonstrate your knowledge.

Assignment Guidelines:

1. **Answer Structure:**
   - For theoretical questions, provide clear and concise explanations.
   - Use examples wherever possible to illustrate your answers.
   - For coding questions, write Java code in a clear and readable format, including comments to explain key sections of the code.

2. Your assignment will be graded based on the following:
   - **Understanding of Concepts:** Accuracy and completeness of answers.
   - **Code Quality:** Code readability, correct syntax, adherence to Java best practices, and clear use of object-oriented principles.
   - **Clarity and Presentation:** Clear explanations, use of examples, proper formatting.

---

1. **Explain the purpose of the `this` keyword in Java.** Give an example where `this` is used to avoid ambiguity.
2. **Describe the concept of inheritance in Java.** How does it support code reuse?
3. **What is the difference between method overloading and method overriding in Java?** Provide an example where each would be useful.
4. **What is the significance of the `super` keyword in Java?** How does it differ from `this`?
5. **Explain single and multilevel inheritance in Java.** Can Java support multiple inheritance for classes? Why or why not?
6. **What are the differences between a constructor and a method in Java?** Can constructors be overloaded? Explain with an example.
7. **Describe polymorphism in Java.** How is polymorphism implemented through method overriding?
8. What is the difference between an instance variable and a static variable in Java?

9. Suppose you have a `Vehicle` class with `start()` and `stop()` methods.How would you design a `Car` and `Bike` class that inherit from `Vehicle` and override the `start()` method to provide specific implementations?

10. Imagine you have a `Library` system with a class `Book`. You need a method to search for books by title, author, or ISBN.How would you implement method overloading for this search functionality?

11. You are building an application that manages employees. Each `Employee` has a name and salary. There are two types of employees: `Manager` and `Developer`. Both types of employees should have a `calculateBonus()` method, but managers get a higher

bonus.How would you implement this structure using inheritance and method overriding?

12. Consider a base class `Animal` with a method `makeSound()`. You have classes `Dog`, `Cat`, and `Cow` that should each implement `makeSound()` with their own unique sounds.Describe how polymorphism and method overriding are applied in this scenario.

13. In a library management system, you have a class `Person` with attributes `name` and `id`. You need to create subclasses `Librarian` and `Member`.How would you use inheritance to design this, ensuring that each subclass has unique behaviors?

14. You have a class `Rectangle` with a method `area()` that calculates the area. You want to create a `Square` class that extends `Rectangle`.How can you ensure that the `Square` class calculates the area correctly, without overriding the `area()` method in the `Rectangle` class?

15. Consider a `Product` class with attributes `name`, `price`, and `discount()`. Now, create a subclass `Electronics` with a specific discount for electronic products.Describe how you would use inheritance and method overriding for the `discount()` method in the subclass.

16. **Create a Java class named `Person` with fields for `name` and `age`.**

    - Add a constructor to initialize these fields.
    - Then create a subclass `Student` that inherits from `Person` and has an additional field for `grade`.
    - Demonstrate using `super` to call the constructor of `Person` from `Student`.

17. **Write a Java program that demonstrates method overloading.** Create a class `Calculator` with methods `add()` that can accept two, three, or four integer parameters and return their sum.
18. **Implement a class `Animal` with a method `sound()`.** Create subclasses `Dog`, `Cat`, and `Bird` that override `sound()` with their specific sounds.
    a. Create an `Animal` array to hold instances of `Dog`, `Cat`, and `Bird`, and use a loop to call `sound()` on each element, demonstrating polymorphism.
19. **Design a base class `Shape` with an `area()` method that returns 0.**
    a. Create subclasses `Circle` and `Rectangle`, each with its own attributes (like radius for `Circle`, and length and width for `Rectangle`).
    b. Override the `area()` method in each subclass to calculate and return the area specific to that shape.
20. **Create a class hierarchy for a `Bank` application.**
    a. Define a base class `BankAccount` with a method `withdraw()`.
    b. Create subclasses `SavingsAccount` and `CheckingAccount`. In `SavingsAccount`, limit the number of withdrawals allowed, and in `CheckingAccount`, add an overdraft limit.
    c. Override `withdraw()` in each subclass to include these specific behaviors.

1. What is the Java Virtual Machine (JVM), and why is it important for Java?
2. Explain the difference between JDK, JRE, and JVM.
3. Describe the purpose of the `public static void main(String[] args)` method in Java.
4. What is the difference between `==` and `.equals()` in Java?

5. List the eight primitive data types in Java and provide examples of each.
6. Explain what type casting is in Java. Provide an example of both implicit and explicit casting.
7. What is a `final` variable in Java? Can it be changed once it's initialized?

8. Explain the difference between `break` and `continue` statements.
9. Write a Java program to find the largest number among three numbers using conditional statements.
10. Describe the `switch` statement and its limitations in Java.

11. Explain the concepts of encapsulation, inheritance, polymorphism, and abstraction in Java with examples.
12. What is method overloading and method overriding? Give an example of each.
13. Explain the use of the `super` keyword in Java. When would you use it?
14. Define a class in Java. What is the purpose of a constructor within a class, and how is it different from a regular method?
15. Explain the concept of "this" keyword in Java. Provide an example of how and why it is used within a class.
16. What are getter and setter methods in Java, and why are they important for encapsulation? Provide an example of each.
17. Write a Java program that creates a `Student` class with fields for name, age, and grade. Then create a method to display the student details.
18. What is inheritance in Java, and why is it useful in object-oriented programming? Provide an example.
19. Explain the difference between single and multilevel inheritance in Java. Can Java support multiple inheritance for classes? Why or why not?
20. Write a Java program that defines a base class `Animal` with a method `makeSound()`. Create a derived class `Dog` that overrides `makeSound()` to print "Bark." Demonstrate polymorphism by creating an `Animal` reference to a `Dog` object.
21. What is method overriding, and how does it differ from method overloading? Explain the significance of the `@Override` annotation in Java.